2d704f8d-395c-450a-b56f-39521d0663dc

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'cocoonstack/cocoon' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/cocoon/cocoon (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-05-15T05:09:05.156+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
fix: prevent orphan VMM via socket tiebreaker + status one-shot default
</pr_title>

<pr_description>
Summary

Closes #48 and #49. Both were triggers for the orphan cocoon/CH processes we found on `cocoon-pool-2` (8 alive CH/FC without DB entries; multiple cocoon-CLI watchers stranded by prior vk-cocoon restarts).

#49 — `vm rm --force` could clean DB while CH stayed alive

\`hypervisor/stop.go:DeleteAll\` treats \`WithRunningVM ⇒ ErrNotRunning\` as "VM already gone, just clean DB". That's correct for the legitimate "VM crashed / cleanly stopped" cases. The leak path is when the pidfile/cmdline check returns a false negative (stale pidfile, recycled PID, manual rundir tampering) — cocoon believes the VM is dead while CH is still listening on its api socket.

Fix:
- Add \`Backend.IsAPISocketLive(rec)\` — a pidfile-independent probe over \`utils.CheckSocket\`.
- Call it as a tiebreaker in \`DeleteAll\` right before \`RemoveVMDirs + DB.Update\`. If the socket answers, refuse the delete.

The tiebreaker fires **unconditionally** (not gated on ErrNotRunning):
- AF_UNIX has no TIME_WAIT and \`TerminateProcess\` only returns nil after full pid reap, so a successful stop guarantees the listener has been torn down — no false positives.
- Firing on the happy path also catches \`utils/pidfd_linux.go:16-17\`'s own \`VerifyProcessCmdline\` false-negative shortcut that otherwise returns \`(true, nil)\` without sending a signal.

#48 — \`vm status\` polling default leaked processes for 21 days

\`cmd/vm/status.go\` always entered a 5s polling loop. From a shell that disconnected without SIGHUP propagation (sudo wrapper, bash -c pipeline, tmux without huponexit), the loop survived indefinitely. We observed 21-day-old orphans of \`sudo cocoon vm status <vmid> --format json | python3 -c 'json.load(sys.stdin)'\`.

Fix:
- Add \`--watch\` flag (default false). Without it, run \`statusOnce\` — single snapshot, exit.
- \`--event\` still implies streaming events (unchanged so vk-cocoon's WatchEvents keeps working).
- Extract \`renderVMList(vms, format)\` so \`Handler.List\` and \`statusOnce\` share the JSON/table/empty-list branch instead of forking.

Breaking change: \`cocoon vm status\` callers who relied on auto-polling must add \`--watch\`. Consciously chosen — the orphan risk far outweighs the convenience.

```
## Test plan

- [x] \`go build ./...\` and \`go test ./...\` green
- [x] \`make lint\` dual-platform 0 issues
- [x] /simplify pass: senior walk + 3 parallel review agents,
all findings addressed
- [ ] Smoke on cocoonset-gke-private after merge:
  - Trigger a rundir / pidfile drift, confirm \`vm rm --force\`
rejects with the new tiebreaker error
  - \`cocoon vm status\` exits immediately; \`cocoon vm status
--watch\` enters the refresh loop as before
</pr_description>

## Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
## Pull request overview

This PR addresses two orphan-process failure modes by (1) adding
a pidfile-independent Unix-socket liveness probe as a last-line
guard before deleting VM records/dirs, and (2) changing `cocoon
vm status` to default to a one-shot snapshot unless explicitly
put into watch/event streaming modes.

**Changes:**
- Add an API-socket connectivity “tiebreaker” check in VM delete
to avoid removing DB entries while a VMM is still alive.
- Change `vm status` default behavior to one-shot output;
introduce `--watch` to opt into the refresh loop (with `--event`
preserved for streaming diffs).
- Refactor VM list rendering/filtering so list + status can
share the same JSON/table output paths.

### Reviewed changes

Copilot reviewed 4 out of 4 changed files in this pull request
and generated 4 comments.

| File | Description |
| ---- | ----------- |
| hypervisor/stop.go | Adds an API socket responsiveness check
```

```
before deleting VM dirs/DB records. |
| hypervisor/state.go | Introduces `Backend.IsAPISocketLive`
helper backed by `utils.CheckSocket`. |
| cmd/vm/status.go | Implements one-shot status default, adds
shared `renderVMList`, and factors filtering into
`applyFilters`. |
| cmd/vm/commands.go | Updates `vm status` CLI help and adds `--
watch` flag/interval wording. |




---

💡 <a>Add Copilot custom instructions</a> for smarter, more
guided reviews. <a href="https://docs.github.com/en/copilot/
customizing-copilot/adding-repository-custom-instructions-for-
github-copilot">Learn how to get started</a>.
</comment_old>

</pr_comments>

<pr_reviews>

<comment_thread_resolved>
<file>hypervisor/state.go:30</file>
<original_commit>702c56b</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
IsAPISocketLive collapses all dial errors into “not live” by
comparing CheckSocket(...) == nil. That means timeouts /
EMFILE / permission errors will be treated as “socket not live”
and DeleteAll will proceed to remove dirs + delete the DB
record, which can reintroduce the orphan leak under load.
Consider returning (bool, error) (or at least logging and
failing closed on timeout/unknown errors) so delete can be
safely aborted when socket liveness is indeterminate.

</comment_old>

<comment_old>
<author>@CMGS</author>
@copilot good catch. Fixed in 967e90d: IsAPISocketLive now
returns true (refuse delete) on any non-{ENOENT,ECONNREFUSED}
dial error, logging the uncertain probe at warn. ENOENT/
```

ECONNREFUSED are the only two outcomes that genuinely mean 'no listener'; anything else (EMFILE under fd exhaustion, transient kernel weirdness, EPERM on hardened socket) is now treated as 'maybe alive' and refuses the delete.
</comment_old>

</comment_thread_resolved>

<comment_thread>
<file>hypervisor/stop.go:67</file>
<original_commit>702c56b</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
The comment says the socket-probe is a tiebreaker when the pidfile said “not running”, but the probe currently runs unconditionally (even after a successful stop). Please update the comment to match the actual behavior so future changes don’t accidentally assume it only runs on ErrNotRunning paths.

</comment_old>

<comment_new>
<comment_id>3246008424</comment_id>
<author>@CMGS</author>
@copilot fixed in 967e90d: comment rewritten to spell out the unconditional behavior (AF_UNIX has no TIME_WAIT so a real stop guarantees no listener, and firing always also catches false-negative pidfile/cmdline shortcuts including the pidfd_linux.go:16-17 path).
</comment_new>

</comment_thread>

<comment_thread>
<file>hypervisor/stop.go:69</file>
<original_commit>702c56b</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
This new error message is the primary user-facing signal for the orphan-vmm safety check, but it doesn’t give a clear remediation. Consider including the VM id/ref and a short next-step hint (e.g., stop the VM again, or manually kill the VMM / investigate the runDir) to reduce operator confusion when deletes are refused.

</comment_old>

<comment_new>
<comment_id>3246008435</comment_id>
<author>@CMGS</author>
@copilot fixed in 967e90d. Error now includes a terse remediation: 'kill the vmm process then retry'. Didn't add VM id to the message because the caller in ForEachVM already prepends 'Delete: item </comment_new>

</comment_thread>

<comment_thread>
<file>cmd/vm/commands.go:174</file>
<original_commit>702c56b</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
The --format flag help text says “event mode only”, but vm status now uses --format in one-shot mode via statusOnce/renderVMList. Please update the flag description to reflect the new behavior (and clarify what happens when --watch is set, since the refresh loop currently always renders a table regardless of --format).

</comment_old>

<comment_new>
<comment_id>3246008450</comment_id>
<author>@CMGS</author>
@copilot fixed in 967e90d: '--format' help is now 'json (one-shot + event modes; --watch always renders a table)' to match the new behavior.
</comment_new>

</comment_thread>

</pr_reviews>

</comments>

The last **1** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement><rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you

address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

<system_reminder>
You have the capability to call multiple tools in a single response.
For maximum efficiency, whenever you need to perform multiple independent operations, ALWAYS call tools simultaneously whenever the actions can be done in parallel rather than sequentially (e.g. multiple reads/edits to different files). Especially when exploring repository, searching, reading files, viewing directories, validating changes. For example, you can read 3 different files in parallel, or edit different files in parallel. However, if some tool calls depend on previous calls to inform dependent values like the parameters, do NOT call these tools in parallel and instead call them sequentially (e.g. reading shell output from a previous command should be sequential as it requires the sessionID).
</system_reminder>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.